Control Barrier Functions with Symmetry under Moving Obstacles.

Rameez Wajid, Bardh Hoxha, Georgios Fainekos and Sriram Sankaranaryanan

***Abstract*— We present a novel approach to synthesizing control barrier functions for nonlinear dynamical systems for avoiding moving obstacles. Our approach distinguishes parts of the state-space of the system as positional states that are part of a workspace. The dynamics are assumed invariant under translations in the workspace. Furthermore, the obstacle position in the workspace is known through periodic measurements. Between these measurements, the obstacles are assumed to move non-deterministically under constraints that allow for a variety of possible obstacle motions. Our work considers a special class of control barrier functions that maintain safety under discrete jumps resulting from the periodic translation in the workspace each time the obstacle position is measured. At the same time, the barrier functions satisfy a controlled decrease condition between measurements. We use Sum-of-Squares (SOS) techniques to synthesize these barrier functions and demonstrate the effectiveness of this approach on some interesting nonlinear dynamical systems.**

I. Introduction

II. Related Work

Control Barrier Functions: [1], [2]

III. Problem Statement

We will now discuss the problem statement and state the key assumptions. Let $\mathbf{x} \in \mathbb{R}^n$ denote a vector of state-variables of a dynamical system that includes quantities such as position, angles, velocities and angular velocities. Let us assume that the state-variables are partitioned as $\mathbf{x} : (\mathbf{y}, \mathbf{z})$ wherein $\mathbf{y} \in \mathbb{R}^m$ are terms workspace variables and $\mathbf{z} \in \mathbb{R}^{n-m}$ are the non-workspace state variables of the system. For instance, consider a vehicle whose states include (x, y, v_x, v_y), representing positions (x, y) and velocities (v_x, v_y). We designate $\mathbf{y} = (x, y)$ as the state-space variables.

The dynamics are given by $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$ for a Lipschitz-continuous function f and control inputs $\mathbf{u} \in U \subseteq \mathbb{R}^k$.

Assumption 1 (Workspace Invariance): Let us assume that the function $f(\mathbf{x}, \mathbf{u}) = g(\mathbf{z}, \mathbf{u})$, or in other words, is independent of the workspace variables $\mathbf{y}$. Therefore, for any two states of the form $\mathbf{x}_1 = (\mathbf{y}_1, \mathbf{z})$ and $\mathbf{x}_2 = (\mathbf{y}_2, \mathbf{z})$, $f(\mathbf{x}_1, \mathbf{u}) = f(\mathbf{x}_2, \mathbf{u})$. This is simply the assumption of translational invariance of the system dynamics.

We illustrate with a simple example below.

Wajid and Sankaranarayanan are with the Department of Computer Science, University of Colorado Boulder, USA. Hoxha and Fainekos are with Toyota Motors, USA.

Example 1: Consider the Dubin's vehicle dynamics with state variables $\mathbf{x} : (x, y, \theta)$ and control input $u \in \mathbb{R}$. The workspace variables are $\mathbf{y} : (x, y)$. The dynamics are given by:

$$\dot{x} = \cos(\theta), \dot{y} = \sin(\theta), \dot{\theta} = u\,.$$

Note that the RHS of the dynamics are independent of the position (x, y) guaranteeing translational symmetry.

a) Obstacle Model: We assume that there is an obstacle $\mathcal{O}$ whose dynamics are defined in the workspace in the form of constraints $g(\mathbf{y}_o, \dot{\mathbf{y}}_o) \geq 0$, wherein $\mathbf{y}_o$ represents the obstacle position and $\dot{\mathbf{y}}_o$ represents the obstacle velocity in the workspace.

Example 2 (Velocity-Bounded Obstacle): We will consider a velocity bounded obstacle with the constraint $\|\dot{\mathbf{y}}_o\| \leq \alpha$, wherein α is the velocity bound.

Given an obstacle with constraints $g(\mathbf{y}_o, \dot{\mathbf{y}}_o) \geq 0$ over its positions and velocities and time $\tau > 0$, let $\hat{\mathcal{O}}(\tau)$ represent the *reachable set* of all obstacle positions within time τ wherein the obstacle dynamics satisfies the constraints given by g at all times:

$$\hat{\mathcal{O}} = \left\{ \mathbf{y}_o(t) \;\middle|\; \begin{array}{c} t \in [0, \tau],\ \mathbf{y}_o(0) = \mathbf{0} \text{ and} \\ \forall t \in [0, \tau] g(\mathbf{y}_o(t), \dot{\mathbf{y}}_o(t)) \geq 0 \end{array} \right\}$$

Example 3: Consider the velocity-bounded obstacle model from Ex. 2. The set $\hat{\mathcal{O}} = \{(x, y) \mid x^2 + y^2 \leq \alpha^2\tau^2\}$ represents all reachable positions within time $\tau > 0$.

b) Measurement Model: Let us assume that the obstacle position is available every $\tau > 0$ time units. For simplicity we will assume that this position measurement is error-free. However, our work readily accommodates erroneous positions by simply defining the reachable set $\hat{\mathcal{O}}$ appropriately.

In our framework, we will place the obstacle at $\mathbf{y}_o = \mathbf{0}$, whenever its position is known to us, by shifting our workspace coordinates of the system from $\mathbf{y}(t)$ to $\mathbf{y}(t) - \mathbf{y}_o$ whenever the obstacle position is measured. This allows us to then move the obstacle to $\mathbf{y}_o(t^+) = \mathbf{0}$.

Definition 1 (Translational Jump): Let us assume that the system receives an obstacle measurement $\mathbf{y}_o(t) = \mathbf{y}_o$ at time $t = k\tau$ for $k \in \mathbb{N}$. The translational jump corresponding to this measurement resets the obstacle position $\mathbf{y}_o(t^+) = \mathbf{0}$ and the system state to $\mathbf{x}(t^+) = (\mathbf{y}(t) - \mathbf{y}_o, \mathbf{z}(t))$ that changes the workspace state while leaving the other state variables unaltered.

Therefore, the overall execution of the system starting from initial state $\mathbf{x}(0)$, initial obstacle position $\mathbf{y}_o(0) = \mathbf{0}$

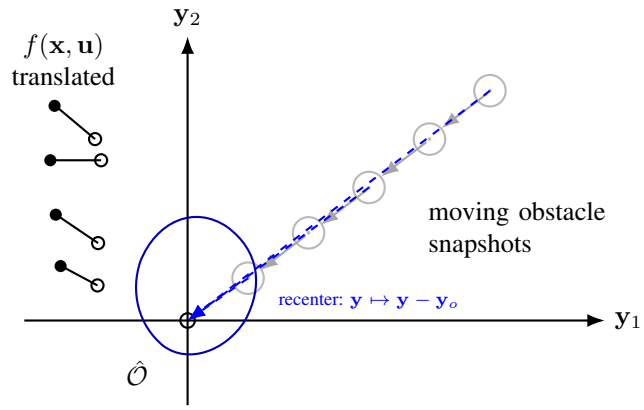


Fig. 1: Translation invariance and recentering for moving obstacles. Each obstacle snapshot is translated back to the origin via $y \mapsto y - y_o$, so the same local feedback template can be reused in relative coordinates.

and control input $\mathbf{u}(t)$ is a function $\mathbf{x} : \mathbb{R}_{\geq 0} \to \mathbb{R}^n$ that maps time t to state $\mathbf{x}(t)$ such that

1) $\mathbf{x}(s)$ is a smooth function in each open interval $s \in (k\tau, (k+1)\tau)$ with $\dot{\mathbf{x}}(s) = f(\mathbf{x}(s), \mathbf{u}(s))$;
2) For each $k \in \mathbb{N}_{>0}$, $\mathbf{x}(k\tau)$ is *left-continuous* i.e, $\lim_{s \to k\tau^-} \mathbf{x}(s)$ exists.
3) For each $k \in \mathbb{N}_{>0}$, $\mathbf{x}(k\tau^+) = \lim_{t \to k\tau^+} = (\mathbf{y}(k\tau) - \mathbf{y}_o(t), \mathbf{z}(k\tau))$, wherein $\mathbf{y}_o(t) \in \hat{\mathcal{O}}$ is the measurement obtained at time $t = k\tau$.

c) Safety Enforcement: A trajectory potentially collides with the obstacle if $\mathbf{x}(t) \in \hat{\mathcal{O}}$ for any time t. We assume that the following information is provided to us:

1) The system dynamics $\mathbf{x} = f(\mathbf{x}, \mathbf{u})$ with control inputs drawn from some compact set U, and satisfying translational invariance over the workspace coordinates $\mathbf{y}$.
2) The measurement time period τ, assuming precise measurement.
3) The obstacle reachable set $\hat{\mathcal{O}}$ consisting of all obstacle reachable configurations during the time interval $[0, \tau]$.

We wish to synthesize a control barrier function $B : \mathbb{R}^n \to \mathbb{R}$, a smooth function that is: (a) strictly positive over the unsafe set $\hat{\mathcal{O}}$; (b) the set $\{\mathbf{x} \mid B(\mathbf{x}) \leq 0\}$ can be maintained control invariant; and (c) control invariant over possible translational jumps resulting from periodic measurements.

In the subsequent section, we will present a series of conditions that will esnure that such a function B can be used inside a *safety filter* to guarantee collision avoidance despite arbitrary obstacle movement within the set $\hat{\mathcal{O}}$.

IV. Repulsive Barrier Functions

Let us recall the definition of a control barrier function (CBF), that is subsumed by the more general condition provided by Ames et al [2].

Definition 2: A smooth function $B(\mathbf{x})$ is a CBF with respect to the dynamics $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$ in a state space $X \subseteq \mathbb{R}^n$ and control inputs $\mathbf{u} \in U \subseteq \mathbb{R}^k$ over an unsafe set X_u iff the following conditions hold: (a) $\forall\ \mathbf{x} \in X_u, B(\mathbf{x}) > 0$; (b) $\forall \mathbf{x} \in X, \exists \mathbf{u} \in U,\ \nabla B \cdot f(\mathbf{x}, \mathbf{u}) \leq \lambda B(\mathbf{x})$, wherein λ is an exponential decrease rate.

Theorem 1: Given a CBF $B(\mathbf{x})$, there exists a feedback law $\kappa : X \to U$ such that for all initial states $\mathbf{x}(0)$ with $B(\mathbf{x}(0)) \leq 0$, the closed loop trajectory of the system $\dot{\mathbf{x}} = f(\mathbf{x}, \kappa(\mathbf{x}))$ satisfies the property $B(\mathbf{x}(t)) \leq \exp(\lambda t) B(\mathbf{x}(0)) \leq 0$.

The proof can be obtained by adapting the more general case shown in Ames et al [2]. However, in our setting, we need to adapt the CBF definition to deal with the expected periodic jumps.

Let $\mathbf{y} \in \mathcal{Y}$ represent all workspace configurations and $\mathbf{z} \in \mathcal{Z}$ represent non-workspace state variables so that the state-space $\mathcal{X} = \mathcal{Y} \times \mathcal{Z}$. Let us write the barrier function $B(\mathbf{x}) = B(\mathbf{y}, \mathbf{z})$.

Definition 3 (Repulsive Barrier Function): A smooth function $B(\mathbf{y}, \mathbf{z})$ is said to be a *repulsive* barrier function w.r.t to unsafe set $\hat{\mathcal{O}}$ and periodic translation jumps iff the following conditions hold:

1) **(Unsafe Set)** $B(\mathbf{y}, \mathbf{z}) > 0$ for all $\mathbf{y} \in \hat{\mathcal{O}}$ and $\mathbf{z} \in \mathcal{Z}$.
2) **(Decrease)** $\forall \mathbf{y} \in \mathcal{Y}, \forall \mathbf{z} \in \mathcal{Z}, \exists \mathbf{u} \in U,\ \nabla B \cdot f(\mathbf{x}, \mathbf{u}) \leq \lambda B(\mathbf{x})$ for $\lambda > 0$. The strict positivity of λ makes this function a *repulsive* function.
3) **(Bounded Jump)** There is a fixed constant $K > 0$ s.t.

$$\forall (\mathbf{y}, \mathbf{z}) \in \mathcal{Y} \times \mathcal{Z}, \forall \mathbf{y}_o \in \hat{\mathcal{O}},\ B(\mathbf{y} - \mathbf{y}_o, \mathbf{z}) - B(\mathbf{y}, \mathbf{z}) \leq K\,.$$

Let $\mathbf{u} = \kappa(\mathbf{x})$ be the safety enforcing feedback law associated B that satisfies the results of Theorem 1. Given a repulsive barrier function B, let us choose a constant $\delta > 0$ as our *margin* and let $S_\delta = \{\mathbf{x} \mid -\delta < B(\mathbf{x}) \leq \mathbf{0}\}$ denote a "safety margin". Note by definition $S_\delta \cap \hat{\mathcal{O}} = \emptyset$ and S_δ is part of the control invariant region. However, we will ensure that *the system never reaches a state in* S_δ. To this end, we define the "repulsion region" $R_{\delta,K} = \{\mathbf{x} \mid -\delta - K \leq B(\mathbf{x}) \leq -\delta\}$.

Lemma 1: Suppose $\lambda \geq \frac{1}{\tau} \log\left(1 + \frac{K}{\delta}\right)$, then whenever $\mathbf{x}(t) \in R_{\delta,K}$, applying the feedback $\mathbf{u} = \kappa(\mathbf{x})$ over $[t, t+\tau]$, ensures that $B(\mathbf{x}((t+\tau)^-)) \leq -\delta - K$.

Proof: Since $\lambda \geq \frac{1}{\tau} \log\left(1 + \frac{K}{\delta}\right)$, we get $\exp(\lambda\tau) \geq 1 + \frac{K}{\delta}$. From Theorem 1, we get $B(\mathbf{x}(t+\tau)) \leq \exp(\lambda\tau) B(\mathbf{x}(t)) \leq \exp(\lambda\tau)(-\delta)$, since $B(\mathbf{x}(t)) \leq -\delta$. Therefore, $B(\mathbf{x}(t+\tau)) \leq -\delta \exp(\lambda\tau) \leq -\delta(1 + \frac{K}{\delta})$. Simplifying, we obtain $B(\mathbf{x}(t+\tau)) \leq -\delta - K$. ■

We now prove that using a repulsive barrier function allows us to find a control strategy that establishes the absence of collision with the moving obstacle.

Theorem 2: Suppose B is a repulsive barrier function with $\lambda \geq \frac{1}{\tau} \log\left(1 + \frac{K}{\delta}\right)$ and the initial state $\mathbf{x}(0)$ is such that $B(\mathbf{x}(0^+)) \leq -\delta$. There exists a control strategy such that the resulting trajectory will never intersect $\hat{\mathcal{O}}$.

Proof: From Def. 3 we can infer that at a given time t a system trajectory intersects $\hat{\mathcal{O}}$, that is, for a collision

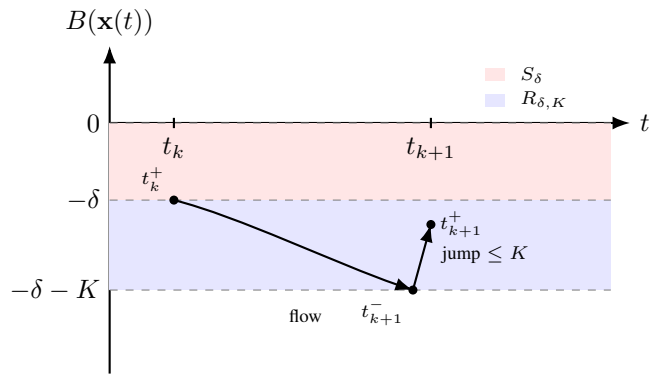


Fig. 2: Sampled-time repulsion under translational jumps. The red band denotes the safety-margin set $S_\delta = \{x \mid -\delta < B(x) \leq 0\}$, and the blue band denotes the repulsion region $R_{\delta,K} = \{x \mid -\delta - K \leq B(x) \leq -\delta\}$. Starting from $B(x(t_k^+)) \leq -\delta$, the flow drives the barrier value to $B(x(t_{k+1}^-)) \leq -\delta - K$, and the translational jump increases it by at most K, yielding $B(x(t_{k+1}^+)) \leq -\delta$.

to occur $B(\mathbf{x}(t)) > 0$. However, given $B(\mathbf{x}(0)) \leq -\delta$, from Lemma 1 we know that when selecting a $\lambda \geq \frac{1}{\tau}\log\left(1+\frac{K}{\delta}\right)$, it ensures a repulsive decrease condition and $B(\mathbf{x}(t+\tau)) \leq -\delta - K$. Since the argument holds for every τ step, whenever the system is in $R_{\delta,K}$, applying the feedback law given by the RBF ensures that the barrier function decreases exponentially and the entire trajectory remains collision-free. ■

In the next section, we describe the procedure for synthesizing repulsive barrier functions using SOS programs.

V. Synthesis of Repulsive Barrier Functions

In this section we will first discuss Sum-of-Squares (SOS) programming and a way to encode our repulsive barrier conditions as SOS. Then will proceed with the discussion on barrier function synthesis.

A. Sum-of-Squares Programming

For a polynomial $\sigma \in \mathbb{R}[x_1, \ldots, x_n]$ to be an SOS polynomial, $\sigma = p_1^2 + \ldots + p_k^2$ for some polynomials $p_1, \ldots, p_k$. Then σ being SOS also makes it non-negative over $\mathbb{R}^n$, although the converse does not hold. This non-negativity property allows us to have SOS programming with the positivstellensatz as a way of obtaining certificates over semi-algebraic domains.

For our barrier synthesis, a key ingredient in exploiting the SOS structure is encoding the conditions in Definition 3 as an SOS programming problem via a positivstellensatz procedure. We'll use the Putinar's Positivstellensatz for our encoding. Using the repulsive barrrier conditions, we get the following SOS certificates:

$$\left.\begin{array}{l} B(\mathbf{y},\mathbf{z}) \equiv \Sigma\alpha_i^y p_i^y(\mathbf{y}) + \Sigma\alpha_j^z p_j^z(\mathbf{z}) + \alpha_0 \\ -\nabla B(\mathbf{y},\mathbf{z}) \cdot f((\mathbf{y},\mathbf{z})), \kappa(\mathbf{y},\mathbf{z})) + \lambda B \equiv \Sigma\beta_i^y q_i^y(\mathbf{y}) + \Sigma\beta_j^z q_j^z(\mathbf{z}) + \beta_0 \\ K - (B(\mathbf{y}-\mathbf{y}_o,\mathbf{z}) - B(\mathbf{y},\mathbf{z})) \equiv \Sigma\sigma_i^y r_i^y(\mathbf{y}) + \Sigma\sigma_j^z r_j^z(\mathbf{z}) + \Sigma\sigma_j^o r_j^o(\mathbf{y}_o) + \sigma_0 \\ \alpha, \beta, \sigma \text{ SOS} \end{array}\right\} \quad (1)$$

The SOS feasibility problem can be solved by framing it as a semi-definite program.

B. Synthesizing Barrier Functions

For synthesizing the repulsive barrier function B, we will solve an SDP based on the SOS problem defined above. The template for B is chosen as a linear combination $\sum_{i=1}^{k} c_i\phi_i(\mathbf{x})$ over monomials $\phi_i(\mathbf{x})$ and unknown coefficients $\mathbf{c} = (c_1, \ldots, c_k)$. Next, we will solve the SOS problem in Eq. 1 with B being the unknown function, as an SDP. If the SDP is feasible, we will get a barrier candidate B. The procedure is presented in Algorithm 1.

Algorithm 1: Algorithm for multiple control barrier computation

Data: System Σ, unsafe set $\hat{\mathcal{O}}$, finite control set U_{fin}, constants $\epsilon > 0$, $\tau > 0$, $K > 0$ and $\delta > 0$, degree bound D, test set S.
Result: Set of barrier functions and associated controls $\mathcal{B}$.

$\mathcal{B} = \emptyset$ /* Initialize to empty set */
$\lambda = \frac{1}{\tau}\log\left(1+\frac{K}{\delta}\right)$ /* Fixed to worst case */
for $\mathbf{u}_t \in U_{\text{fin}}$ **do**
  for $\mathbf{x}_t \in S$ **do**
    find polynomial B of degree D
      s.t. $(\forall \mathbf{y} \in \hat{\mathcal{O}}, \forall \mathbf{z} \in \mathcal{Z})\ B(\mathbf{y},\mathbf{z}) \geq \epsilon$
      $(\forall \mathbf{y} \in \mathcal{Y}, \forall \mathbf{z} \in \mathcal{Z})\ \nabla B \cdot (f(\mathbf{x},\mathbf{u}_t)) \leq \lambda B$
      $(\forall (\mathbf{y},\mathbf{z}) \in \mathcal{Y} \times \mathcal{Z}, \forall \mathbf{y}_o \in \hat{\mathcal{O}})\ B(\mathbf{y}-\mathbf{y}_o,\mathbf{z}) - B(\mathbf{y},\mathbf{z}) \leq K$
      $B(\mathbf{x}_t) \leq 0$
    if B *found* **then**
      $\mathcal{B} = \mathcal{B} \cup \{(B, \mathbf{u}_t)\}$
      $S = S \setminus \{\mathbf{x} \in S \mid B(\mathbf{x}) \leq 0\}$
        /* remove already eliminated test point */
return $\mathcal{B}$

In the algorithm, we begin by assuming a finite set of control inputs $U_{\text{fin}} \in U$ for our system and a set of sampled test states S. The choice U_{fin} is discussed in detail later in this section. Next, we iterate over the control inputs $\mathbf{u}_t \in U_{\text{fin}}$ and the test points $\mathbf{x}_t \in S$. In each iteration, we search for a barrier candidate based on the SOS problem formulated earlier. Once we find a successful barrier candidate, we add it to our collection of repulsive barriers, eliminate all the test points it satisfies, and move on to the next iteration. In this way, we are able to compute multiple repulsive barrier functions, thus effectively covering a large part of the workspace for our system.

C. Selecting Control Laws

Now we discuss the selection of a finite set of control laws that would guarantee that all resulting trajectories for the system never intersect $\hat{\mathcal{O}}$.

Theorem 3: If B is a smooth barrier function, satisfying the conditions of Def. 3, there exists a finite set $U_{\text{fin}} \subseteq U$ such that for every $\mathbf{y} \in \mathcal{Y}, \mathbf{z} \in \mathcal{Z}$ there exists $\mathbf{u} \in U_{\text{fin}}$ such that $\nabla B \cdot (f(\mathbf{x}, \mathbf{u})) \leq \lambda B$.

Proof: Given B is smooth, for all $\mathbf{x}$ the function $\nabla B \cdot (f(\mathbf{x}, \kappa(\mathbf{x}))) \leq \lambda B$ is also smooth. Hence, for any $\mathbf{x}$ and any $\mathbf{u} \in U$, there exists a Lipschitz constant $\mathcal{L} > 0$ such that $\nabla B \cdot (f(\mathbf{x}, \mathbf{u})) - \nabla B \cdot (f(\mathbf{x}, \kappa(\mathbf{x})) \leq \mathcal{L}\|\mathbf{u} - \kappa(\mathbf{x})\|$. Assuming that we choose U_{fin} as a $\varepsilon/(2\mathcal{L})$-packing of the set U so that for all $\mathbf{u} \in U$, there exists a $\mathbf{u}_f \in U_{\text{fin}}$ such that $\|\mathbf{u} - \mathbf{u}_f\| \leq \varepsilon/(2\mathcal{L})$. Now we can choose $\hat{\mathbf{u}} \in U_{\text{fin}}$ such that $\|\hat{\mathbf{u}} - \kappa(\mathbf{x})\| \leq \varepsilon/(2\mathcal{L})$. Finally,

$$\begin{aligned} \nabla B \cdot (f(\mathbf{x}, \mathbf{u})) &\leq \nabla B \cdot (f(\mathbf{x}, \kappa(\mathbf{x})) + \mathcal{L}\|\mathbf{u} - \kappa(\mathbf{x})\| \\ &\leq -\xi + L\xi/(2L) \ \leq -\xi/2\,. \end{aligned}$$

■

Remark 1: If the set U is a polyhedral set and f is a control affine function (which vehicle models generally are), then the set U_{fin} can be chosen to be the set of vertices of U.

The above theorem guarantees that if there exists a control strategy that satisfies a barrier certificate, a control strategy that switches between inputs in $\hat{\mathbf{u}}$ will also provide the same guarantee.

VI. Safety Controller

VII. Implementation and Experiments

A. 3D Dynamics (Dubin's Model)

We now consider a coordinated turn model with 5 state variables and 2 control input $u_1 \in [-5, 5]$.

$$\begin{aligned} \dot{x}_1 &= v \cos x_3, \\ \dot{x}_2 &= v \sin x_3, \\ \dot{x}_3 &= u_1. \end{aligned}$$

The unsafe set is taken to be $\hat{\mathcal{O}} = \{(x_1, x_2) | x_1^2 + x_2^2 \leq 1\}$. The set U_{fin} was set to the vertices of $U : [-5, 5]^2$ and we chose 100 randomly selected test states.

B. 5D Dynamics (Coordinated Turn Model)

We now consider a coordinated turn model with 5 state variables and 2 control inputs $u_1, u_2 \in [-5, 5]^2$.

$$\begin{aligned} \dot{x}_1 &= x_3 \cos x_4, & \dot{x}_4 &= x_5, \\ \dot{x}_2 &= x_3 \sin x_4, & \dot{x}_5 &= u_2, \\ \dot{x}_3 &= u_1, \end{aligned}$$

The unsafe set is taken to be $\hat{\mathcal{O}} = \{(x_1, x_2) | x_1^2 + x_2^2 \leq 1\}$. The set U_{fin} was set to the vertices of $U : [-5, 5]^2$ and we chose 500 randomly selected test states.

C. 6D Nonlinear Dynamics (Planar Multirotor)

We now consider a planar multirotor model with 6 state variables and 2 control inputs $u_1, u_2 \in [0, 2]^2$.

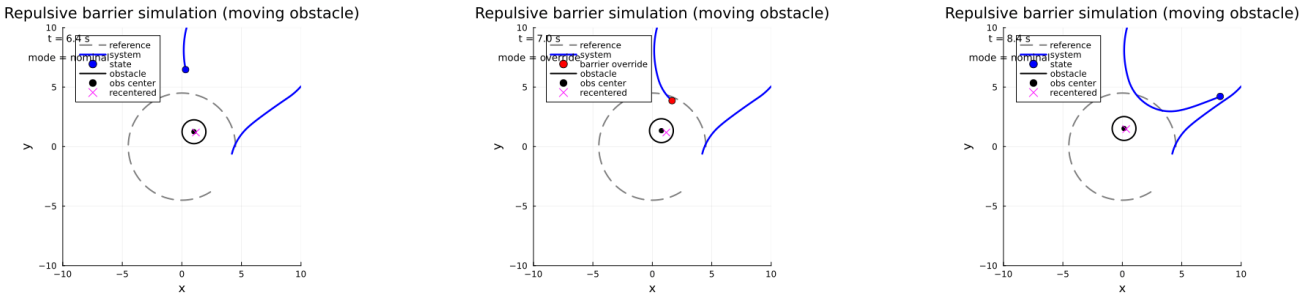
$$\begin{aligned} \dot{x}_1 &= x_3, & \dot{x}_4 &= (u_1 + u_2) \cos x_5 - 2, \\ \dot{x}_2 &= x_4, & \dot{x}_5 &= x_6, \\ \dot{x}_3 &= (u_1 + u_2) \sin x_5, & \dot{x}_6 &= u_1 - u_2, \end{aligned}$$

The unsafe set is taken to be $\hat{\mathcal{O}} = \{(x_1, x_2) | x_1^2 + x_2^2 \leq 1\}$. The set U_{fin} was set to the vertices of $U : [0, 2]^2$ and we chose 900 randomly selected test states.

References

[1] P. Wieland and F. Allgöwer, "Constructive safety using control barrier functions," *IFAC Proceedings Volumes*, vol. 40, no. 12, pp. 462–467, 2007.

[2] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control barrier functions: Theory and applications," in *European Control Conference (ECC)*, pp. 3420–3431, 2019.



(a) **(b)** **(c)**

Fig. 3: Collision avoidance for a moving obstacle using repulsive barrier functions for 3D Dubin's vehicle model from Section VII-A where **(a)** the vehicle approaches the moving obstacle using nominal controller; **(b)** the control override kicks in (indicated by the vehicle going red); **(c)** back to nominal controller after obstacle avoidance.